

Late Breaking Results: Fine-Tuning LLMs for Test Stimuli Generation

Hyeonwoo Park, Seonghyeon Park, and Seokhyeong Kang
Pohang University of Science and Technology, Pohang, Republic of Korea
{jimmy0709, seonghyeon98, shkang}@postech.ac.kr

Abstract—The understanding and reasoning capabilities of large language models (LLMs) with text data have made them widely used for test stimuli generation. Existing studies have primarily focused on methods such as prompt engineering or providing feedback to the LLMs’ generated outputs to improve test stimuli generation. However, these approaches have not been successful in enhancing the LLMs’ domain-specific performance in generating test stimuli. In this paper, we introduce a framework for fine-tuning LLMs for test stimuli generation through dataset generation and reinforcement learning (RL). Our dataset generation approach creates a table-shaped test stimuli dataset, which helps ensure that the LLM produces consistent outputs. Additionally, our two-stage fine-tuning process involves training the LLMs on domain-specific data and using RL to provide feedback on the generated outputs, further enhancing the LLMs’ performance in test stimuli generation. Experimental results confirm that our framework improves syntax correctness and code coverage of test stimuli, outperforming commercial models.

I. INTRODUCTION

Recently, large language models (LLMs) have been used to generate testbenches by leveraging their text generation capabilities [3]. A testbench should cover a wide range of possible design behaviors, and to achieve high code coverage, it is necessary to generate test stimuli¹ that can evaluate all the states of the design. To generate high-coverage test stimuli, existing studies [3], [8], [9] have used LLMs through prompt engineering involving iterative generation. Despite recent advancements, LLMs-based testbench generation still faces several challenges. First, if the model has limitations in hardware design knowledge, prompt engineering alone cannot address these inherent shortcomings, even with iterations. Second, evaluating the quality of the generated test stimuli, such as code coverage (including Line, Condition, FSM, etc.), is challenging, as LLMs cannot assess the coverage performance of their own generated outputs. Last, using commercial models like GPT introduces data privacy concerns and limitations for in-depth research [4].

In this paper, we address these challenges and propose methods to enhance the model’s performance further. Our key contributions are outlined as follows:

- **Fine-tuning Dataset Generation** Motivated by [6], we introduce a table-shaped test stimuli dataset, which allows LLMs to generate test stimuli with a high syntax correctness and code coverage compared to other dataset types.
- **2-Stage Fine-Tuning Framework** We propose a two-stage fine-tuning framework for LLMs. The first stage involves su-

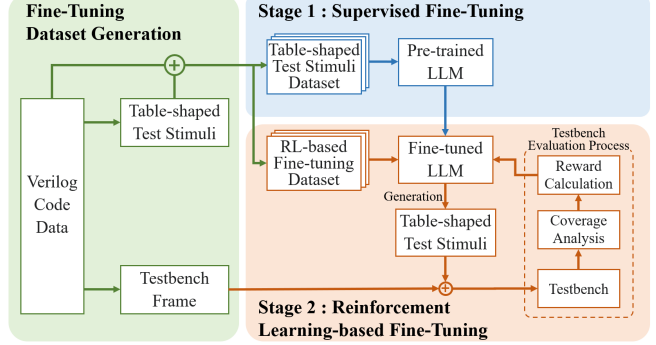


Fig. 1: Overall framework.

pervised fine-tuning (SFT), followed by reinforcement learning (RL)-based fine-tuning. Compared to our baseline model, our framework improves syntax correctness by 50.1% and coverage by 30.8%, with the fine-tuned model outperforming the commercial model, GPT, by 6.2% in coverage.

II. METHODOLOGY

Figure 1 illustrates our proposed framework. To perform fine-tuning, we first generate a table-shaped test stimuli dataset from Verilog code data. Next, we apply SFT using this dataset to fine-tune the LLMs. Finally, we conduct RL-based fine-tuning with coverage analysis.

A. Table-Shaped Test Stimuli Dataset Generation

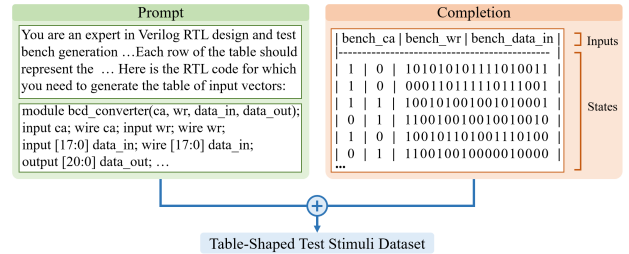


Fig. 2: Table-shaped test stimuli dataset.

The SFT dataset consists of prompts and their corresponding completions². Figure 2 shows our table-shaped test stimuli dataset, which includes Verilog code as prompts and a table of test stimuli as completions. The first row of the table contains the inputs from the Verilog code, listed in the order of their declaration, while the remaining rows represent the states of all inputs in binary form. The binary values are randomly generated. We design the dataset with random values, as the goal of SFT is to focus on generating structured completions rather than high-coverage test stimuli. Fine-tuning with structured data helps the LLMs produce consistent outputs, improving syntax correctness rather than generating testbench directly.

Prompts and completions are the inputs and outputs used in fine-tuning.

This work was supported by the Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.2022-0-01172) and Korea Institute of Advancement of Technology (KIAT) grant funded by the Korea Government (MOTIE) (RS-2024-00401466).

In this paper, testbench refers to the Verilog used to test the design, while test stimuli refers to the input provided to the design.

B. 2-Stage Fine-Tuning

Our fine-tuning framework consists of two stages. First, we perform SFT using the table-shaped test stimuli dataset. SFT fine-tunes the model on domain-specific data, enabling the generation of structured outputs with high syntax correctness. Second, we perform RL-based fine-tuning to allow the model to generate high-coverage test stimuli. We use the proximal policy optimization algorithm [5] for RL. Using the testbench frame, a testbench without input stimuli created from Verilog data, along with the LLM-generated test stimuli, we create the testbench and evaluate its code coverage. The coverage results are presented as the average of observable code coverage types, including Line, Condition, Toggle, FSM, and Branch [2]. Based on coverage results, we define reward as follows:

$$reward = \begin{cases} -1 & \text{if } fail_{syntax} \\ coverage & \text{otherwise} \end{cases} \quad (1)$$

We then use this reward for policy updates. For RL-based fine-tuning, we only use prompts, which consist of Verilog code and randomly generated table-shaped test stimuli. With iterative reward feedback, RL-based fine-tuning enables LLMs to generate high coverage test stimuli.

III. EXPERIMENTAL RESULTS

A. Experimental Setup

Our framework is implemented in *Python* and the baseline model is *deepseek-coder-6.7b-instruct* [1]. We collect Verilog codes from GitHub repositories and verify its syntax and synthesizability. For code coverage and syntax check, we use *Synopsys VCS v17.12* [10] and *Synopsys Verdi v20.03* [11], respectively. For evaluation, we use the *pass@k* metric [3], calculated based on results generated 20 times. We evaluate the model on 92 designs out of 171 designs from Autochip [7], excluding designs that failed syntax checks or had no input or outputs. All datasets and scripts are available on GitHub repository [12].

B. Table-Shaped Test Stimuli Dataset Evaluation

TABLE I: Evaluation of Test Stimuli Dataset and Fine-Tuning

Experiment	Models	Autochip Benchmark [7]			Coverage (%)
		k=1	k=5	k=10	
Dataset (Section 3.B)	D _{testbench}	51.6	52.4	52.8	39.5
	D _{vector}	83.6	89.2	90.1	53.8
	D _{table}	96.6	96.7	96.7	56.1
2-Stage Fine-Tuning (Section 3.C)	Baseline _{testbench} [1]	34.8	53.5	54.8	67.1
	Baseline _{table} [1]	46.6	74.8	83.7	57.0
	GPT-4o _{testbench}	48.7	51.6	53.2	81.1
	GPT-4-turbo _{testbench}	51.3	52.7	54.9	81.6
	GPT-4o _{table}	44.2	48.7	53.3	76.3
	GPT-4-turbo _{table}	45.3	49.8	53.3	69.4
	Random	—	—	—	61.3
	D _{table} &RL	96.7	98.1	98.4	87.8

The best result in each column is highlighted in **bold** for each experiment.

To validate our proposed table-shaped test stimuli dataset (D_{table}), we create two additional datasets (D_{testbench} and D_{vector}).

D_{testbench} has completions as a testbench with randomly generated input stimuli, which is commonly used in prompt engineering. D_{vector} contains test stimuli represented as binary values on each line rather than in a table. Each dataset is used to fine-tune the baseline model through SFT. As shown in Table I, the syntax correctness of the proposed dataset outperforms both D_{testbench} and D_{vector} by 45.0% and 13.0%, 44.3% and 7.5%, and 43.9% and 6.5% at k = 1, 5, 10, respectively. A reduction in coverage after fine-tuning is observed, stemming from the use of a random test stimuli dataset, which is subsequently addressed through RL-based fine-tuning.

C. 2-Stage Fine-Tuning Evaluation

To evaluate our fine-tuning framework, we compare the results with the baseline model and state-of-the-art commercial LLMs (GPT-4-turbo and GPT-4o). Prompt engineering involves designing inputs to guide the model's output, which operates differently from fine-tuning. Therefore, it is excluded from the comparison group. We assess the test stimuli generation capabilities of these LLMs by generating two types of outputs: (1) testbench and (2) table-shaped test stimuli. Table I shows that our framework outperforms the baseline_{table} model, improving syntax correctness by 50.1%, 23.3%, and 14.7% at k = 1, 5, 10, respectively, and coverage by 30.8%. Compared to commercial LLMs, our framework shows superior syntax correctness, achieving 45.4%, 45.4%, and 43.5% higher in syntax correctness at k = 1, 5, 10, respectively, along with a 6.2% higher coverage. This demonstrates that our fine-tuning framework enables LLMs to generate test stimuli with both higher syntax correctness and coverage. Additionally, when compared with random test stimuli (Random), generated with the same length as those from the fine-tuned LLMs, we show that the fine-tuned model achieves 26.5% higher coverage.

IV. CONCLUSION

In this paper, we propose the first fine-tuning dataset and framework for test stimuli generation. Our table-shaped test stimuli dataset facilitates effective learning during SFT. Our 2-stage fine-tuning framework optimizes LLMs for test stimuli generation, achieving performance that surpasses commercial models in terms of code coverage and syntax correctness.

REFERENCES

- [1] D. Guo, Q. Zhu, D. Yang, *et al.*, "DeepSeek-Coder: When the Large Language Model Meets Programming-The Rise of Code Intelligence", *arXiv*, 2024, pp. 1-23.
- [2] J. Y. Jou, CN. J. Liu, "Coverage analysis techniques for hdl design validation" *Proc. Asia Pacific Chip Design Languages*, 1999, pp. 48-55.
- [3] R. Qiu, G.L. Zhang, R. Drechsler, *et al.*, "Autobench: Automatic Testbench Generation and Evaluation Using llms for HDL Design" *Proc. MLCAD*, 2024, pp. 1-10.
- [4] P.P. Ray, "ChatGPT: A Comprehensive Review on Background, Applications, Key Challenges, Bias, Ethics, Limitations" *arXiv*, 2023, pp. 1-8.
- [5] J. Schulman, F. Wolski, P. Dhariwal *et al.*, "Proximal Policy Optimization Algorithms", *arXiv*, 2017, pp. 1-12.
- [6] Y. Sui, M. Zhou, M. Zhou, *et al.*, "Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study" *Proc. WSDM*, 2024, pp. 645-654.
- [7] S. Thakur, J. Blocklove, H. Pearce, *et al.*, "AutoChip: Automating HDL Generation Using LLM Feedback" *arXiv*, 2023, pp. 1-7.
- [8] C. Xiao, Y. Deng, Z. Yang, *et al.*, "LLM-based Processor Verification: A Case Study for Neuromorphic Processor" *Proc. DATE*, 2024, pp. 1-6.
- [9] Z. Zhang, G. Chadwick, H. McNally *et al.*, "LLM4DV: Using Large Language Models For Hardware Test Stimuli Generation" *arXiv*, 2023, pp. 1-7.
- [10] Synopsys VCS User Guide, <http://www.synopsys.com>
- [11] Synopsys Verdi User Guide, <http://www.synopsys.com>
- [12] Fine-tuning-LLM-for-test-stimuli-gen, <https://github.com/kangnam4123/Fine-tuning-LLM-for-test-stimuli-gen>